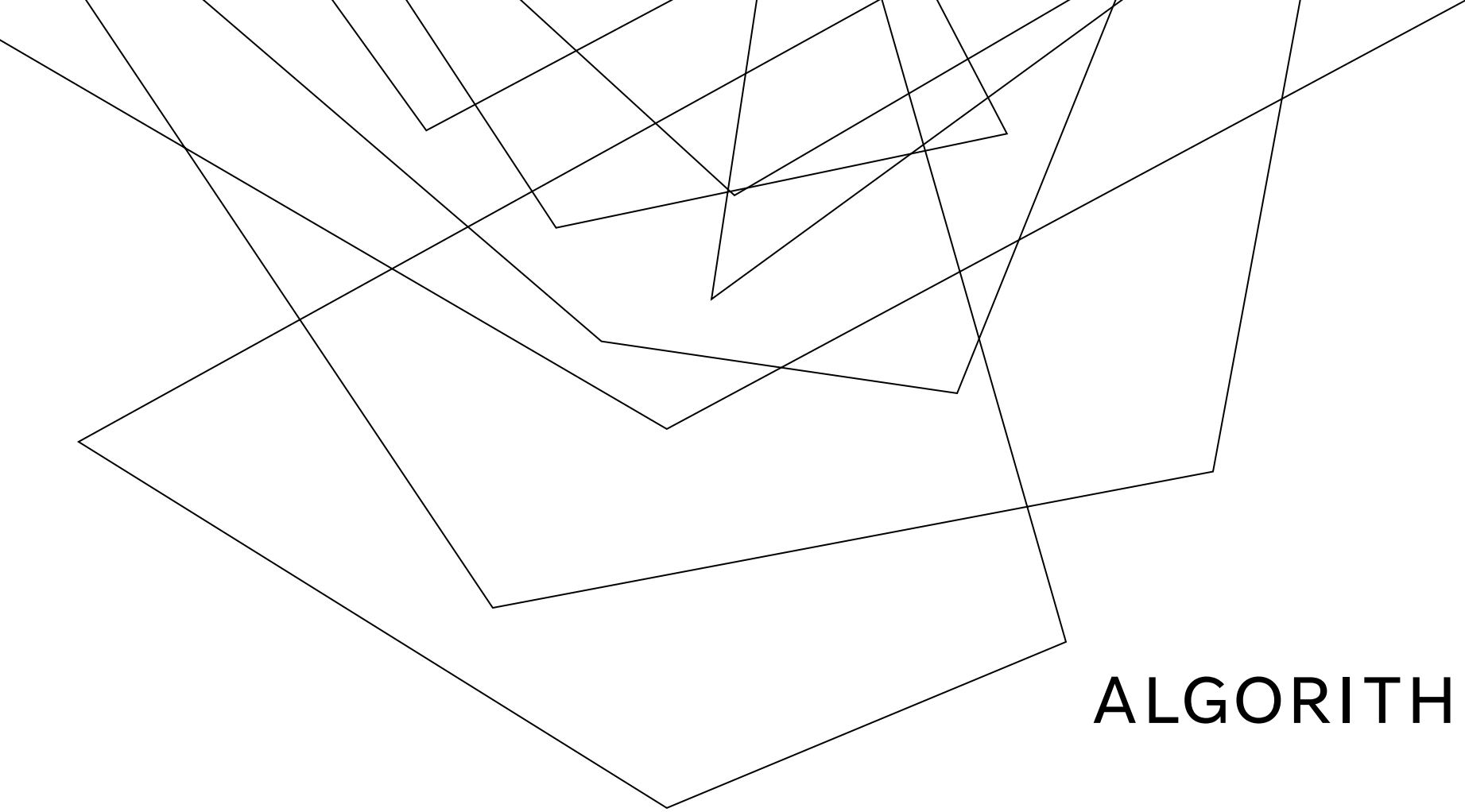


A series of thin, black, overlapping lines forming various geometric shapes like triangles and polygons, creating a complex, abstract pattern in the upper left portion of the slide.

ALGORITHMS

10. DYNAMIC PROGRAMMING (2)





지난 시간 복습

DYNAMIC PROGRAMMING (1)

1. Dynamic Programming
2. Matrix Path Problem
3. Matrix Multiplication Order Problem





목차

목차

1. Longest Common Sub-sequence Problem
2. $2 \times n$ Tiling Problem
3. Making it 1 Problem





LONGEST COMMON SUB-SEQUENCE PROBLEM

LONGEST COMMON SUB-SEQUENCE PROBLEM

두 문자열이 주어졌을 때, 두 문자열의 **최장 공통 부분 문자열(Longest Common Sub-sequence)**의 **길이**를 계산하세요.

어떤 문자열 X의 **부분 문자열(Sub-sequence)**은 X의 문자 일부를 순서를 유지한 채 나열한 문자열이다. 예를 들어, X= “**ABCBDAB**”의 부분 문자열로는 “**BCDB**”가 있다.

어떤 문자열 Z가 다른 두 문자열 X와 Y의 부분 문자열이면, Z는 X와 Y의 **공통 부분 문자열(Common Sub-sequence)**이다. 예를 들어, Z=“**BCA**” 는 X=“**ABCBDAB**” 와 Y=“**BDCABA**”의 공통 부분 문자열이다.

문자열 X와 Y의 **최장 공통 부분 문자열(Longest Common Sub-sequence)**은 X와 Y의 공통 부분 문자열 중 가장 긴 문자열이다.



LONGEST COMMON SUB-SEQUENCE PROBLEM

- Naïve solution

- 두 문자열 x, y 의 길이가 각각 m, n 이고 $m < n$ 이라 가정
- x 의 **모든 부분 문자열을** 전수조사하여 y 의 부분 문자열인지 확인,
그 중 가장 긴 부분 문자열의 길이를 반환
 - x 의 부분 문자열 수 = 2^m
 - y 의 부분 문자열인지 확인하는 시간 = n
- Naive Solution의 시간 복잡도는 $O(n \cdot 2^m)$
 - m 이 클 경우 사실상 처리 불가



LONGEST COMMON SUB-SEQUENCE PROBLEM

Q) Dynamic Programming으로 풀기에 적합한 문제인가?

- Q1) 최적 부분 구조를 갖는가? (= 문제를 재귀적으로 표현할 수 있는가?)
- Q2) 중복 호출로 심각한 비효율이 발생하는가?



LONGEST COMMON SUB-SEQUENCE PROBLEM

Q1) 최적 부분 구조를 갖는가? (= 문제를 재귀적으로 표현할 수 있는가?)

- (case 1) X와 Y의 마지막 문자가 **같을 때**

$$\text{LCS}(X, Y) = \text{LCS}(X[: -1], Y[: -1]) + 1$$

- (case 2) X와 Y의 마지막 문자가 **다를 때**

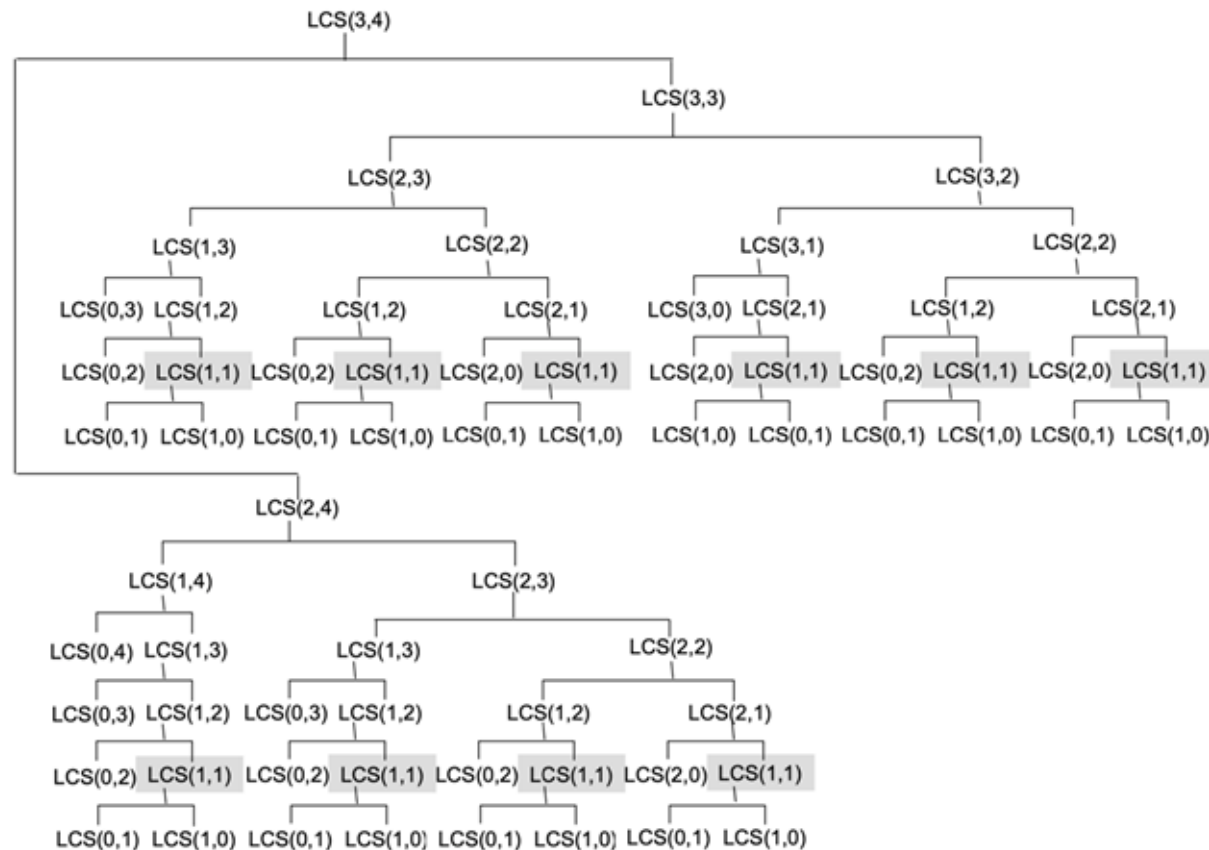
$$\text{LCS}(X, Y) = \max(\text{LCS}(X[: -1], Y), \text{LCS}(X, Y[: -1]))$$



LONGEST COMMON SUB-SEQUENCE PROBLEM

Q2) 중복 호출로 심각한 비효율이 발생하는가?

➔ 함수 재귀 호출 횟수가 지수적으로 증가



LCS	LCS(1, 1) 호출 횟수
LCS(2,2)	2
LCS(2,3)	3
LCS(3,3)	6
LCS(3,4)	10
LCS(4,4)	20
LCS(4,5)	35
LCS(5,5)	70
LCS(5,6)	126
LCS(6,6)	252
LCS(6,7)	462
LCS(7,7)	924



LONGEST COMMON SUB-SEQUENCE PROBLEM

- Bottom-up 방식 (Tabulation) 코드

```
def lcs(X, Y):  
    m, n = len(X), len(Y)  
  
    dp = [[0] * (n+1) for _ in range(m+1)]  
  
    for i in range(m):  
        for j in range(n):  
            if X[i] == Y[j]: # case 1  
                dp[i][j] = dp[i-1][j-1] + 1  
            else: # case 2  
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])  
  
    return dp[m-1][n-1]
```

dp

	0	1	2	3	4	5
0	---	---	---	---	---	→
1	---	---	---	---	---	→
2	---	---	---	---	---	→
3						
4						
5						



LONGEST COMMON SUB-SEQUENCE PROBLEM

- Top-down 방식 (Memoization) 코드

```
def lcs(X, Y):  
    m, n = len(X), len(Y)  
  
    dp = [[-1] * n for _ in range(m)]  
  
    def S(i, j):  
        if i < 0 or j < 0: return 0  
        if dp[i][j] != -1: return dp[i][j]  
  
        if X[i] == Y[j]: # case 1  
            dp[i][j] = S(i-1, j-1) + 1  
        else: # case 2  
            dp[i][j] = max(S(i-1, j), S(i, j-1))  
  
        return dp[i][j]  
  
    return S(m-1, n-1)
```





2XN TILING PROBLEM

2XN TILING PROBLEM

문제

$2 \times n$ 크기의 직사각형을 1×2 , 2×1 타일로 채우는 방법의 수를 구하는 프로그램을 작성하시오.

아래 그림은 2×5 크기의 직사각형을 채운 한 가지 방법의 예이다.



입력

첫째 줄에 n 이 주어진다. ($1 \leq n \leq 1,000$)

출력

첫째 줄에 $2 \times n$ 크기의 직사각형을 채우는 방법의 수를 10,007로 나눈 나머지를 출력한다.

예제 입력 1 복사

2

예제 출력 1 복사

2

예제 입력 2 복사

9

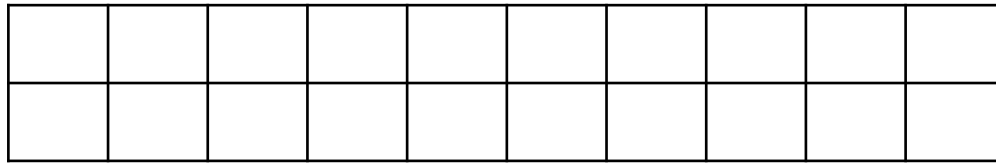
예제 출력 2 복사

55



2XN TILING PROBLEM

q) 최적 부분 구조를 갖는가? (= 문제를 재귀적으로 표현할 수 있는가?)

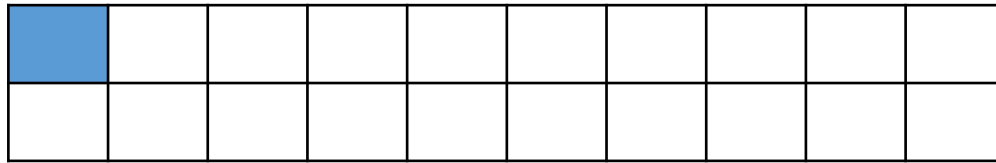


$dp[n] = 2 \times n$ 크기의 직사각형을 채우는 방법의 수



2XN TILING PROBLEM

Q) 최적 부분 구조를 갖는가? (= 문제를 재귀적으로 표현할 수 있는가?)



$dp[n]$ = $2 \times n$ 크기의 직사각형을 채우는 방법의 수

Hint) 파란색 칠한 (1,1) 위치를 채울 수 있는 경우의 수는 두 가지

i) 2×1 타일로 채움 , ii) 1×2 타일로 채움





MAKING IT 1
PROBLEM

MAKING IT 1 PROBLEM

문제

정수 X 에 사용할 수 있는 연산은 다음과 같이 세 가지 이다.

1. X 가 3으로 나누어 떨어지면, 3으로 나눈다.
2. X 가 2로 나누어 떨어지면, 2로 나눈다.
3. 1을 뺀다.

정수 N 이 주어졌을 때, 위와 같은 연산 세 개를 적절히 사용해서 1을 만들려고 한다. 연산을 사용하는 횟수의 최솟값을 출력하시오.

입력

첫째 줄에 1보다 크거나 같고, 10^6 보다 작거나 같은 정수 N 이 주어진다.

출력

첫째 줄에 연산을 하는 횟수의 최솟값을 출력한다.

예제 입력 1 [복사](#)

2

예제 입력 2 [복사](#)

10

예제 출력 1 [복사](#)

1

예제 출력 2 [복사](#)

3



MAKING IT 1 PROBLEM

Q) 최적 부분 구조를 갖는가? (= 문제를 재귀적으로 표현할 수 있는가?)

$dp[i]$ = i 를 1로 만들기 위해 필요한 연산 사용 횟수의 최솟값



MAKING IT 1 PROBLEM (2)

문제

정수 X 에 사용할 수 있는 연산은 다음과 같이 세 가지 이다.

1. X 가 3으로 나누어 떨어지면, 3으로 나눈다.
2. X 가 2로 나누어 떨어지면, 2로 나눈다.
3. 1을 뺀다.

정수 N 이 주어졌을 때, 위와 같은 연산 세 개를 적절히 사용해서 1을 만들려고 한다. 연산을 사용하는 횟수의 최소값을 출력하시오.

입력

첫째 줄에 1보다 크거나 같고, 10^6 보다 작거나 같은 자연수 N 이 주어진다.

출력

첫째 줄에 연산을 하는 횟수의 최소값을 출력한다.

둘째 줄에는 N 을 1로 만드는 방법에 포함되어 있는 수를 공백으로 구분해서 순서대로 출력한다. 정답이 여러 가지인 경우에는 아무거나 출력한다.

예제 입력 1 [복사](#)

2

예제 출력 1 [복사](#)

1
2 1

예제 입력 2 [복사](#)

10

예제 출력 2 [복사](#)

3
10 9 3 1



MAKING IT 1 PROBLEM (2)

- 역추적 (traceback) 을 위해 추가 정보를 저장할 테이블이 필요
- Bottom-up 방식 (Tabulation) 이 직관적

값 테이블

1	2	3	4	5	6	7	8
0							

경로 저장용
테이블

1	2	3	4	5	6	7	8
0							



MAKING IT 1 PROBLEM (2)

- Bottom-up 방식 (Tabulation) 으로 풀 경우

값 테이블

1	2	3	4	5	6	7	8
0	1						

경로 저장용
테이블

1	2	3	4	5	6	7	8
0	1						



MAKING IT 1 PROBLEM (2)

- Bottom-up 방식 (Tabulation) 으로 풀 경우

값 테이블

1	2	3	4	5	6	7	8
0	1	1					

경로 저장용
테이블

1	2	3	4	5	6	7	8
0	1	1					



MAKING IT 1 PROBLEM (2)

- Bottom-up 방식 (Tabulation) 으로 풀 경우

값 테이블

1	2	3	4	5	6	7	8
0	1	1	2				

경로 저장용
테이블

1	2	3	4	5	6	7	8
0	1	1	2				



MAKING IT 1 PROBLEM (2)

- Bottom-up 방식 (Tabulation) 으로 풀 경우

값 테이블

1	2	3	4	5	6	7	8
0	1	1	2	3			

경로 저장용
테이블

1	2	3	4	5	6	7	8
0	1	1	2	4			



MAKING IT 1 PROBLEM (2)

- Bottom-up 방식 (Tabulation) 으로 풀 경우

값 테이블

1	2	3	4	5	6	7	8
0	1	1	2	3	2	3	3

경로 저장용
테이블

1	2	3	4	5	6	7	8
0	1	1	2	4	3	6	4



An abstract graphic on a light gray background. Two thin, dark gray lines intersect. One line is nearly vertical, starting from the top center and extending towards the bottom right. The other line is nearly horizontal, starting from the top left and extending towards the middle right. The intersection point is located in the upper-left quadrant of the image. To the right of this intersection, the word "SUMMARY" is written in a bold, black, sans-serif font.

SUMMARY

SUMMARY

1. Longest Common Sub-sequence Problem
2. $2 \times n$ Tiling Problem
3. Making it 1 Problem



NEXT LECTURE

Greedy algorithm



Two thin, dark gray lines intersect diagonally on a light gray background. One line runs from the top-left towards the bottom-right, and the other runs from the top-right towards the bottom-left. They cross each other in the upper-left quadrant of the image.

QUESTIONS?